

Delta Lake Management & Operations Task

Introduction

This report documents the implementation of a comprehensive Delta Lake pipeline for managing e-commerce order data using [PySpark](#) and [Databricks](#). It covers table creation, data manipulation (insert, update, delete, merge), schema enforcement, schema evolution, transaction history, and time travel.

Requirement 1 — Create the Delta Table

Description: Created the initial orders Delta table with the specified schema and inserted the initial set of 5 order records.

Code:

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DateType, TimestampType, DoubleType
from datetime import datetime

schema = StructType([
    StructField("order_id", IntegerType(), True),
    StructField("customer_id", IntegerType(), True),
    StructField("product_id", IntegerType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("price", DoubleType(), True),
    StructField("status", StringType(), True),
    StructField("order_date", DateType(), True),
    StructField("updated_at", TimestampType(), True)
])

initial_data = [
    (1001, 501, 2001, 2, 100.0, "Pending", datetime.strptime("2026-09-15", "%Y-%m-%d").date(), datetime.strptime("2026-09-15 10:00:00", "%Y-%m-%d %H:%M:%S")),
    (1002, 502, 2002, 1, 250.0, "Shipped", datetime.strptime("2026-09-15", "%Y-%m-%d").date(), datetime.strptime("2026-09-15 11:00:00", "%Y-%m-%d %H:%M:%S")),
    (1003, 503, 2003, 3, 75.0, "Delivered", datetime.strptime("2026-09-16", "%Y-%m-%d").date(), datetime.strptime("2026-09-16 09:30:00", "%Y-%m-%d %H:%M:%S")),
    (1004, 504, 2004, 1, 500.0, "Pending", datetime.strptime("2026-09-16", "%Y-%m-%d").date(), datetime.strptime("2026-09-16 12:00:00", "%Y-%m-%d %H:%M:%S")),
    (1005, 505, 2005, 4, 50.0, "Shipped", datetime.strptime("2026-09-17", "%Y-%m-%d").date(), datetime.strptime("2026-09-17 14:00:00", "%Y-%m-%d %H:%M:%S"))
]

df_initial = spark.createDataFrame(initial_data, schema=schema)

df_initial.write.format("delta").mode("overwrite").saveAsTable("orders")

display(spark.sql("SELECT * FROM orders"))
```

Cell output:

df_initial: pyspark.sql.connect.dataframe.DataFrame = [order_id: integer, customer_id: integer ... 6 more fields]

Table +

	order_id	customer_id	product_id	quantity	price	status	order_date	updated_at
1	1001	501	2001	2	100	Pending	2026-09-15	2026-09-15T10:00:00.000+00:00
2	1002	502	2002	1	250	Shipped	2026-09-15	2026-09-15T11:00:00.000+00:00
3	1003	503	2003	3	75	Delivered	2026-09-16	2026-09-16T09:30:00.000+00:00
4	1004	504	2004	1	500	Pending	2026-09-16	2026-09-16T12:00:00.000+00:00
5	1005	505	2005	4	50	Shipped	2026-09-17	2026-09-17T14:00:00.000+00:00

5 rows 25.171s runtime Refreshed 5m ago

Requirement 2 — Insert New Data

Description: Appended a new batch of orders (order IDs 1006 and 1007) into the Delta table and verified that the total row count became 7.

Code & output:

```

new_data = [
    (1006, 506, 2006, 2, 120.0, "Pending", datetime.strptime("2026-09-18", "%Y-%m-%d").date(), datetime.strptime("2026-09-18 10:00:00", "%Y-%m-%d %H:%M:%S")),
    (1007, 507, 2007, 1, 300.0, "Pending", datetime.strptime("2026-09-18", "%Y-%m-%d").date(), datetime.strptime("2026-09-18 11:00:00", "%Y-%m-%d %H:%M:%S"))
]

df_new = spark.createDataFrame(new_data, schema=schema)
df_new.write.format("delta").mode("append").saveAsTable("orders")

print("Total rows:", spark.sql("SELECT COUNT(*) FROM orders").collect()[0][0])

```

Total rows: 7

Requirement 3 — Update Existing Orders

Description: Updated the status of existing orders: order 1001 to 'Shipped' and order 1004 to 'Cancelled'.

Code & output:

```

from delta.tables import DeltaTable

deltaTable = DeltaTable.forName(spark, "orders")

deltaTable.update(
    condition="order_id IN (1001, 1004)",
    set={
        "status": "CASE WHEN order_id = 1001 THEN 'Shipped' ELSE 'Cancelled' END"
    }
)

display(spark.sql("SELECT * FROM orders WHERE order_id IN (1001, 1004)"))

```

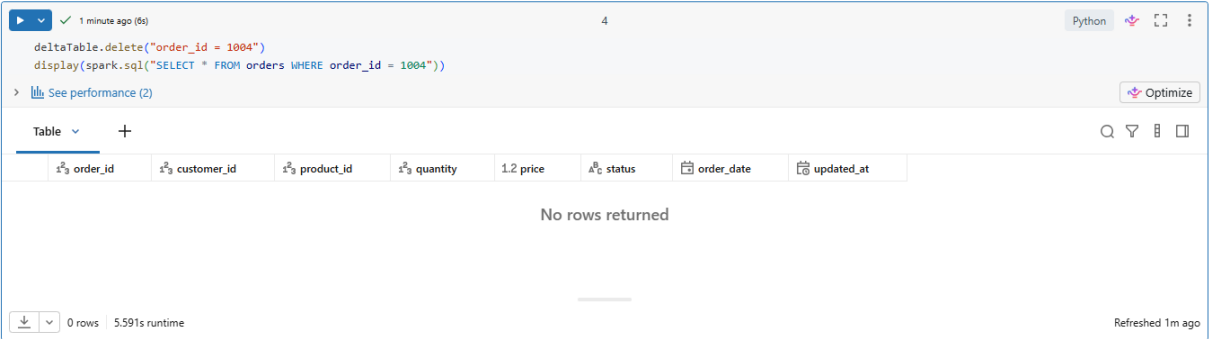
	order_id	customer_id	product_id	quantity	price	status	order_date	updated_at
1	1001	501	2001	2	100	Shipped	2026-09-15	2026-09-15T10:00:00.000+00:00
2	1004	504	2004	1	500	Cancelled	2026-09-16	2026-09-16T12:00:00.000+00:00

2 rows 9.508s runtime Refreshed Now

Requirement 4 — Delete Data

Description: Removed cancelled orders (order_id = 1004) from the operational orders table.

Code:



The screenshot shows a Databricks notebook interface. The code cell contains the following Python code:

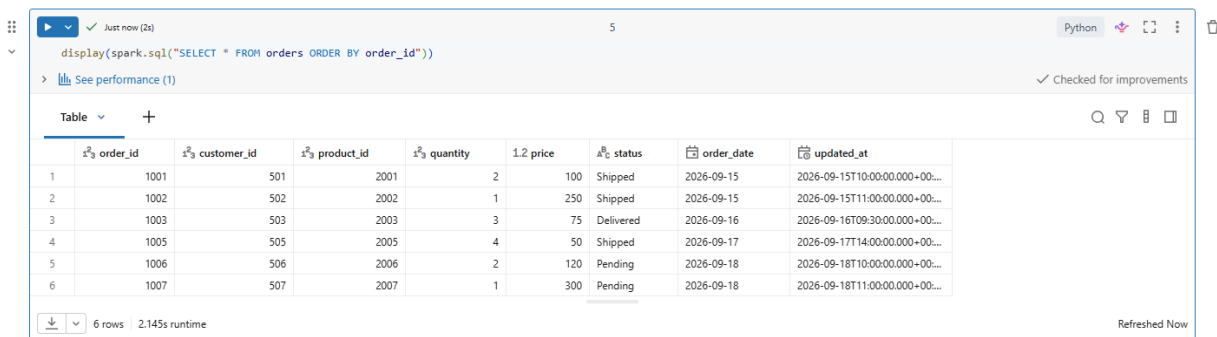
```
deltaTable.delete("order_id = 1004")
display(spark.sql("SELECT * FROM orders WHERE order_id = 1004"))
```

Below the code, the table view shows "No rows returned". The table schema is as follows:

order_id	customer_id	product_id	quantity	price	status	order_date	updated_at
----------	-------------	------------	----------	-------	--------	------------	------------

At the bottom, it indicates "0 rows" and "5.591s runtime".

Verifying that row 4 deleted



The screenshot shows a Databricks notebook interface. The code cell contains the following Python code:

```
display(spark.sql("SELECT * FROM orders ORDER BY order_id"))
```

Below the code, the table view shows 6 rows. The table schema is as follows:

order_id	customer_id	product_id	quantity	price	status	order_date	updated_at
1	1001	501	2001	2	100	Shipped	2026-09-15T10:00:00.000+00:00
2	1002	502	2002	1	250	Shipped	2026-09-15T11:00:00.000+00:00
3	1003	503	2003	3	75	Delivered	2026-09-16T09:30:00.000+00:00
4	1005	505	2005	4	50	Shipped	2026-09-17T14:00:00.000+00:00
5	1006	506	2006	2	120	Pending	2026-09-18T10:00:00.000+00:00
6	1007	507	2007	1	300	Pending	2026-09-18T11:00:00.000+00:00

At the bottom, it indicates "6 rows" and "2.145s runtime".

Requirement 5 — MERGE (Upsert)

Description: Processed a batch containing updates for existing orders (1002 and 1003) and a new record (1008) using the MERGE operation.

Code :

```

) Interrupt 00:05
6 Python
batch_data = [
    (1002, 502, 2002, 1, 250.0, "Delivered", datetime.strptime("2026-09-15", "%Y-%m-%d").date(), datetime.strptime("2026-09-19 09:00:00", "%Y-%m-%d %H:%M:%S")),
    (1003, 503, 2003, 5, 75.0, "Delivered", datetime.strptime("2026-09-16", "%Y-%m-%d").date(), datetime.strptime("2026-09-19 10:00:00", "%Y-%m-%d %H:%M:%S")),
    (1008, 508, 2008, 2, 90.0, "Pending", datetime.strptime("2026-09-19", "%Y-%m-%d").date(), datetime.strptime("2026-09-19 11:00:00", "%Y-%m-%d %H:%M:%S"))
]

df_batch = spark.createDataFrame(batch_data, schema=schema)

deltaTable.alias("target").merge(
    df_batch.alias("source"),
    "target.order_id = source.order_id"
).whenMatchedUpdate(set={
    "quantity": "source.quantity",
    "status": "source.status",
    "updated_at": "source.updated_at"
}).whenNotMatchedInsert(values={
    "order_id": "source.order_id",
    "customer_id": "source.customer_id",
    "product_id": "source.product_id",
    "quantity": "source.quantity",
    "price": "source.price",
    "status": "source.status",
    "order_date": "source.order_date",
    "updated_at": "source.updated_at"
}).execute()

display(spark.sql("SELECT * FROM orders ORDER BY order_id"))
> See performance Statement 0/1 (1) | Tasks 8/8 (0 running)

```

Output:

> df_batch: pyspark.sql.connect.dataframe.DataFrame = [order_id: integer, customer_id: integer ... 6 more fields]

	order_id	customer_id	product_id	quantity	price	status	order_date	updated_at
1	1001	501	2001	2	100	Shipped	2026-09-15	2026-09-15T10:00:00.000+00:00...
2	1002	502	2002	1	250	Delivered	2026-09-15	2026-09-19T09:00:00.000+00:00...
3	1003	503	2003	5	75	Delivered	2026-09-16	2026-09-19T10:00:00.000+00:00...
4	1005	505	2005	4	50	Shipped	2026-09-17	2026-09-17T14:00:00.000+00:00...
5	1006	506	2006	2	120	Pending	2026-09-18	2026-09-18T10:00:00.000+00:00...
6	1007	507	2007	1	300	Pending	2026-09-18	2026-09-18T11:00:00.000+00:00...
7	1008	508	2008	2	90	Pending	2026-09-19	2026-09-19T11:00:00.000+00:00...

7 rows 9.876s runtime

Requirement 6 — Schema Enforcement

Description: Tested schema enforcement by attempting to insert a record with an invalid data type (quantity = "two" instead of integer).

Code & Answers:

```
Just now (2s) 7 Python
try:
    bad_data = [(1009, 509, 2009, "two", 100.0, "Pending", datetime.strptime("2026-09-19", "%Y-%m-%d").date(), datetime.strptime("2026-09-19 12:00:00", "%Y-%m-%d %H:%M:%S"))]
    bad_schema = StructType([
        StructField("order_id", IntegerType(), True),
        StructField("customer_id", IntegerType(), True),
        StructField("product_id", IntegerType(), True),
        StructField("quantity", StringType(), True),
        StructField("price", DoubleType(), True),
        StructField("status", StringType(), True),
        StructField("order_date", DateType(), True),
        StructField("updated_at", TimestampType(), True)
    ])
    df_bad = spark.createDataFrame(bad_data, schema=bad_schema)
    df_bad.write.format("delta").mode("append").saveAsTable("orders")
except Exception as e:
    print("Error caught successfully:", e)

> See performance (1) 1 ✓ Checked for improvements
> df_bad: pyspark.sql.connect.dataframe.DataFrame = [order_id: integer, customer_id: integer ... 6 more fields]
Error caught successfully: [CAST_INVALID_INPUT] The value 'two' of the type "STRING" cannot be cast to "INT" because it is malformed. Correct the value as per the syntax, or change its target type. Use 'try_cast' to tolerate malformed input and return NULL instead. SQLSTATE: 22018

JVM stacktrace:
org.apache.spark.SparkNumberFormatException
    at org.apache.spark.sql.errors.QueryExecutionErrors$.invalidInputInCastToNumberError(QueryExecutionErrors.scala:205)
    at org.apache.spark.sql.catalyst.util.UTF8StringUtil$.withException(UTF8StringUtil.scala:57)
    at org.apache.spark.sql.catalyst.util.UTF8StringUtil$.toIntExact(UTF8StringUtil.scala:34)
    at org.apache.spark.sql.catalyst.expressions.Cast.$anonfun$castToInt$2(Cast.scala:1520)
    at org.apache.spark.sql.catalyst.expressions.Cast.$anonfun$castToInt$2$adapted(Cast.scala:1520)
    at org.apache.spark.sql.catalyst.expressions.Cast.buildCast(Cast.scala:1118)
    at org.apache.spark.sql.catalyst.expressions.Cast.$anonfun$castToInt$1(Cast.scala:1520)
    at org.apache.spark.sql.catalyst.expressions.Cast.nullSafeEval(Cast.scala:2024)
    at org.apache.spark.sql.catalyst.expressions.UnaryExpression.eval(Expression.scala:823)
```

What happened? Delta Lake rejected the insert operation and threw an exception.

Why did Delta Lake reject the record? Because the incoming data types did not match the defined schema of the target Delta table.

What problem does schema enforcement prevent? It prevents data corruption and ensures data quality by blocking invalid or mistyped data from entering the storage layer.

Requirement 7 — Schema Evolution

Description: Modified the table schema to introduce a new column (discount DOUBLE) and inserted a new order containing a discount value.

Code:

```
discount_data = [(1009, 509, 2009, 1, 200.0, "Pending", datetime.strptime("2026-09-19", "%Y-%m-%d").date(), datetime.strptime("2026-09-19 12:00:00", "%Y-%m-%d %H:%M:%S"), 20.0)]

discount_schema = StructType([
    StructField("order_id", IntegerType(), True),
    StructField("customer_id", IntegerType(), True),
    StructField("product_id", IntegerType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("price", DoubleType(), True),
    StructField("status", StringType(), True),
    StructField("order_date", DateType(), True),
    StructField("updated_at", TimestampType(), True),
    StructField("discount", DoubleType(), True)
])

df_discount = spark.createDataFrame(discount_data, schema=discount_schema)

df_discount.write.format("delta") \
    .mode("append") \
    .option("mergeSchema", "true") \
    .saveAsTable("orders")

display(spark.sql("SELECT * FROM orders"))
```

df_discount: pyspark.sql.connect.dataframe.DataFrame = [order_id: integer, customer_id: integer ... 7 more fields]

Output:

order_id	customer_id	product_id	quantity	price	status	order_date	updated_at	discount
1	1001	501	2	100	Shipped	2026-09-15	2026-09-15T10:00:00.000+00:00	null
2	1005	505	4	50	Shipped	2026-09-17	2026-09-17T14:00:00.000+00:00	null
3	1006	506	2	120	Pending	2026-09-18	2026-09-18T10:00:00.000+00:00	null
4	1007	507	1	300	Pending	2026-09-18	2026-09-18T11:00:00.000+00:00	null
5	1002	502	1	250	Delivered	2026-09-15	2026-09-19T09:00:00.000+00:00	null
6	1003	503	5	75	Delivered	2026-09-16	2026-09-19T10:00:00.000+00:00	null
7	1008	508	2	90	Pending	2026-09-19	2026-09-19T11:00:00.000+00:00	null
8	1009	509	1	200	Pending	2026-09-19	2026-09-19T12:00:00.000+00:00	20

Requirement 8 — Table History

Description: Queried the transaction log history of the Delta table to review all operations performed (CREATE, INSERT, UPDATE, DELETE, MERGE).

Code & Answer:

```
spark.sql("DESCRIBE HISTORY orders").show(truncate=False)
```

version	timestamp	userId	userName	operation	operationParameters
8	[2026-09-24 00:23:29]	[76794188406174]	[shereenmohamed632006@gmail.com]	[WRITE]	[{mode -> Append, statsOnLoad -> false, partitionBy -> [], canMergeSchema -> true}]
7	[2026-09-24 00:19:17]	[76794188406174]	[shereenmohamed632006@gmail.com]	[OPTIMIZE]	[{clusterBy -> [], zOrderBy -> [], batchId -> 0, predicate -> [], auto -> true}]
6	[2026-09-24 00:19:14]	[76794188406174]	[shereenmohamed632006@gmail.com]	[MERGE]	[{predicate -> ["(order_id#13666 = order_id#13698)"], clusterBy -> [], matchedP}]
5	[2026-09-24 00:14:33]	[76794188406174]	[shereenmohamed632006@gmail.com]	[OPTIMIZE]	[{clusterBy -> [], zOrderBy -> [], batchId -> 0, predicate -> [], auto -> true}]
4	[2026-09-24 00:14:30]	[76794188406174]	[shereenmohamed632006@gmail.com]	[DELETE]	[{predicate -> ["(order_id#13061 = 1004)"]}]
3	[2026-09-24 00:11:42]	[76794188406174]	[shereenmohamed632006@gmail.com]	[OPTIMIZE]	[{clusterBy -> [], zOrderBy -> [], batchId -> 0, predicate -> [], auto -> true}]
2	[2026-09-24 00:11:39]	[76794188406174]	[shereenmohamed632006@gmail.com]	[UPDATE]	[{predicate -> ["order_id#12299 IN (1001,1004)"]}]
1	[2026-09-24 00:10:47]	[76794188406174]	[shereenmohamed632006@gmail.com]	[WRITE]	[{mode -> Append, statsOnLoad -> true, partitionBy -> []}]
0	[2026-09-24 00:03:16]	[76794188406174]	[shereenmohamed632006@gmail.com]	[CREATE OR REPLACE TABLE AS SELECT]	[{isV1SaveAsTableOverwrite -> true, partitionBy -> [], clusterBy -> [], descrip}]

Why is the Delta transaction log important for maintaining the history of a table? The transaction log (`_delta_log`) records every atomic transaction as ordered JSON files, providing full auditability, enabling time travel, and ensuring ACID compliance.

Requirement 9 — Time Travel

Description: Used Delta Lake Time Travel to query an older version of the table (Version 0) and compared it with the current version to identify deleted records.

Code:

Just now (3s)10Python

```
df_v0 = spark.read.format("delta").option("versionAsOf", 0).table("orders")
df_current = spark.read.format("delta").table("orders")

print("Orders present in Version 0 but deleted in Current version:")
display(df_v0.join(df_current, "order_id", "left_anti"))
```

> 2 dataframes: df_current, df_v0

Orders present in Version 0 but deleted in Current version:

Table

order_id	customer_id	product_id	quantity	price	status	order_date	updated_at	
1	1004	504	2004	1	500	Pending	2026-09-16	2026-09-16T12:00:00.000+00:...